BARUCH COLLEGE

Department of Mathematics • Master of Financial Engineering

DEEP LEARNING IN FINANCE | Lecture Series

Attention & Transformers

Part I

Oualid Missaoui

Adjunct Faculty, Baruch College

Roadmap

From Recurrence to Attention

The Self-Attention Mechanism

Learning & Scaling Attention

The Transformer Model

Training, Inference & LLMs

Philosophy

You can agree with Kant or disagree with him, but you can't get past him.



Immanuel Kant

AI

Use Transformers or don't; no well-rounded modern AI scientist gets to sidestep them.



Ashish Vaswani

Roadmap

From Recurrence to Attention

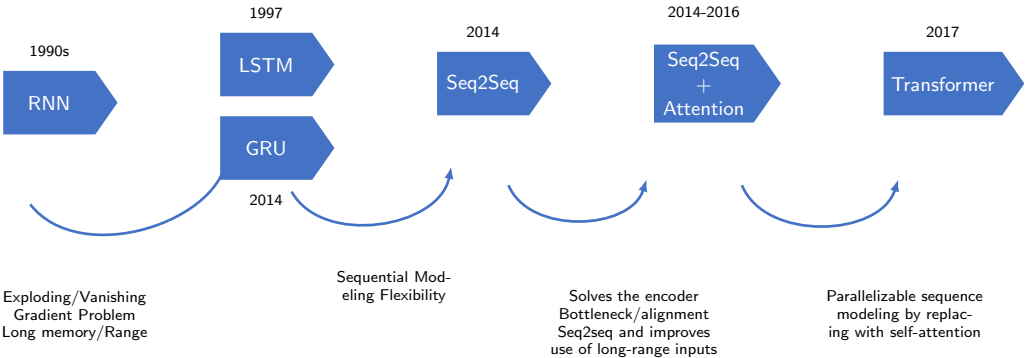
The Self-Attention Mechanism

Learning & Scaling Attention

The Transformer Model

Training, Inference & LLMs

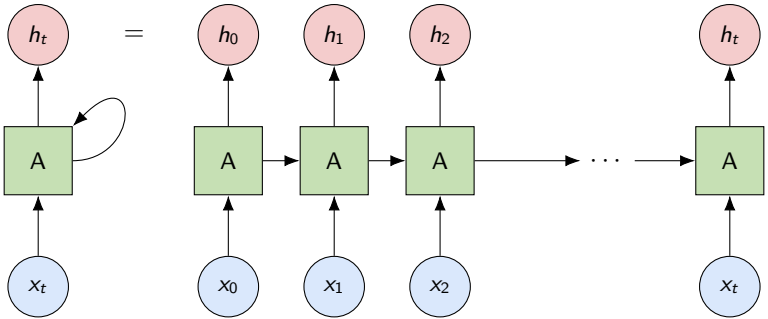
Historical development



Recurrent Neural Networks (RNN)

History:

- John Hopfield (1982), Neural Networks and Physical Systems with Emergent Collective Computational Abilities
- David Rumelhart, G. Hinton and R. J. Williams (1986), Learning Internal Representations by Error Propagation



$$\sigma(x) = \frac{1}{1+e^{-x}}, \qquad \frac{d(\sigma(x))}{dx} = \sigma(x)\big(1-\sigma(x)\big)$$

LSTM vs GRU

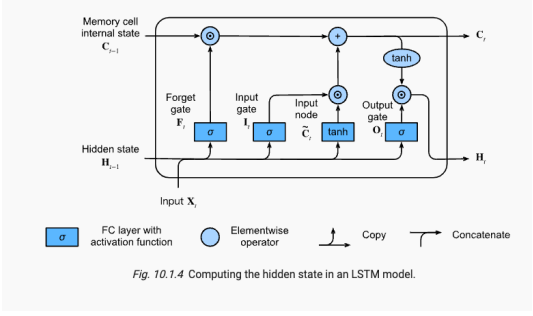


Fig. 10.1.4 Computing the hidden state in an LSTM model.

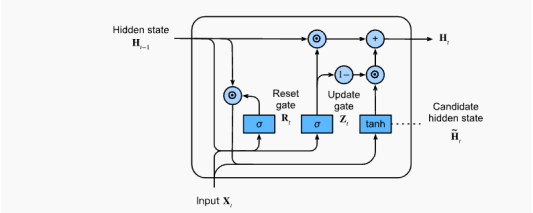
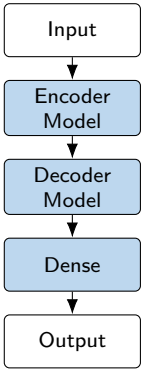


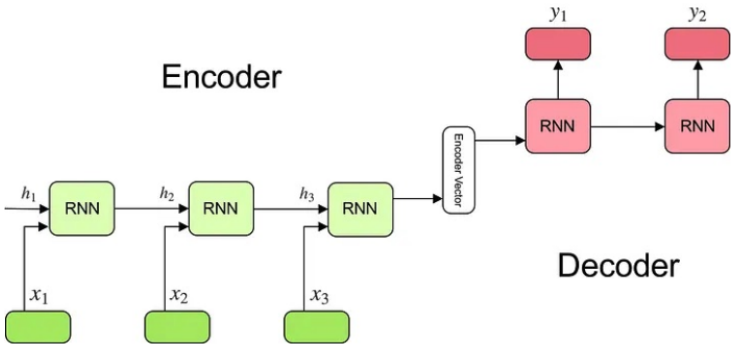
Fig. 10.2.3 Computing the hidden state in a GRU model.

Aston Zhang, Zachary Lipton, Mu Li, and Alex Smola (2023) *Dive into Deep Learning*

Seq2Seq Architecture



Encoder-Decoder LSTM Model Architecture



Encoding an input sentence into a fixed-size vector for the decoder

- A limitation of the architecture is that it encodes the input sequence to a fixed length internal representation.
- This imposes limits on the length of input sequences that can be reasonably learned and results in worse performance for very long input sequences.

The need for attention

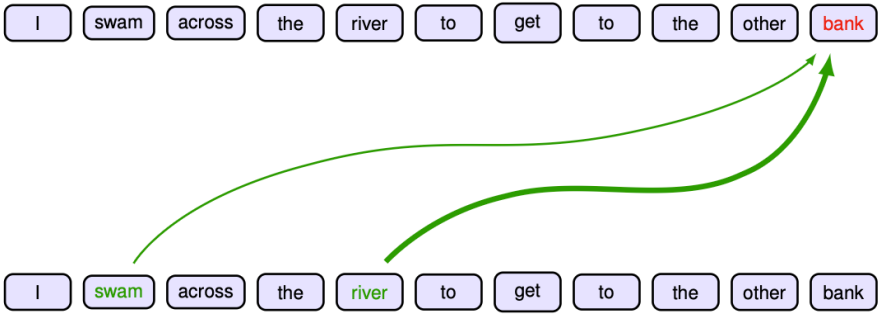


Figure 12.1 Schematic illustration of attention in which the interpretation of the word 'bank' is influenced by the words 'river' and 'swam', with the thickness of each line being indicative of the strength of its influence.

Figure 12.1 Schematic illustration of attention in which the interpretation of the word 'bank' is influenced by the words 'river' and 'swam', with the thickness of each line being indicative of the strength of its influence.

Seq2Seq with Attention (Bahdanau)

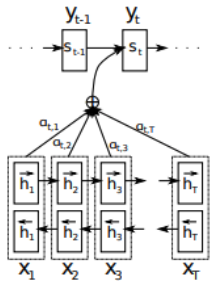


Figure 1: The graphical illustration of the proposed model trying to generate the t-th target word y_t given a source sentence $(x_1, x_2, \ldots, x_T)$.

NEURAL MACHINE TRANSLATION BY JOINTLY LEARNING TO ALIGN AND TRANSLATE

Dzmitry Bahdanau
Jacobs University Bremen, Germany

KyungHyun Cho **Yoshua Bengio***
Université de Montréal

ABSTRACT

Neural machine translation is a recently proposed approach to machine translation. Unlike the traditional statistical machine translation, the neural machine translation aims at building a single neural network that can be jointly tuned to maximize the translation performance. The models proposed recently for neural machine translation often belong to a family of encoder–decoders and encode a source sentence into a fixed-length vector from which a decoder generates a translation. In this paper, we conjecture that the use of a fixed-length vector is a bottleneck in improving the performance of this basic encoder–decoder architecture, and propose to extend this by allowing a model to automatically (soft-)search for parts of a source sentence that are relevant to predicting a target word, without having to form these parts as a hard segment explicitly. With this new approach, we achieve a translation performance comparable to the existing state-of-the-art phrase-based system on the task of English-to-French translation. Furthermore, qualitative analysis reveals that the (soft-)alignments found by the model agree well with our intuition.

Dzmitry Bahdanau et al, *Neural Machine Translation by jointly learning to align and translate*

Seq2Seq: fixed vs dynamic context (c)

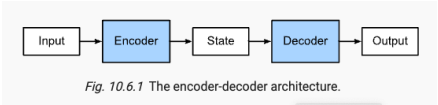


Fig. 10.6.1 The encoder-decoder architecture.

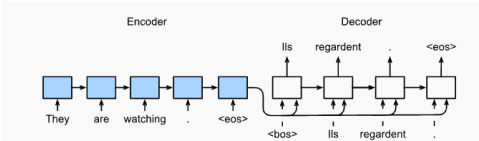


Fig. 10.7.1 Sequence to sequence learning with an RNN encoder and an RNN decoder.

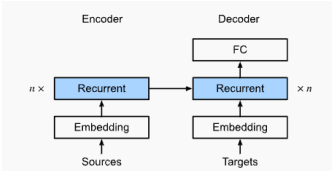


Fig. 10.7.2 Layers in an RNN encoder-decoder model.

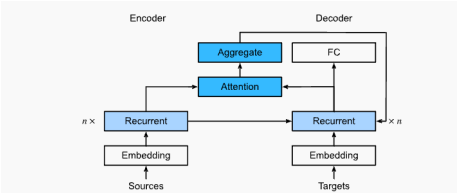


Fig. 11.4.2 Layers in an RNN encoder-decoder model with the Bahdanau attention mechanism.

The key idea is that instead of keeping the context variable c summarizing the source sentence as fixed, we dynamically update it, as a function of both:

1. the original text (encoder hidden states)
2. the text already generated (decoder hidden states)

$$\mathbf{c}_{t'} = \sum_{t=1}^{T} \alpha(\mathbf{s}_{t'-1}, \mathbf{h}_t)\, \mathbf{h}_t.$$

Roadmap

From Recurrence to Attention

The Self-Attention Mechanism

Learning & Scaling Attention

The Transformer Model

Training, Inference & LLMs

Transformers

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

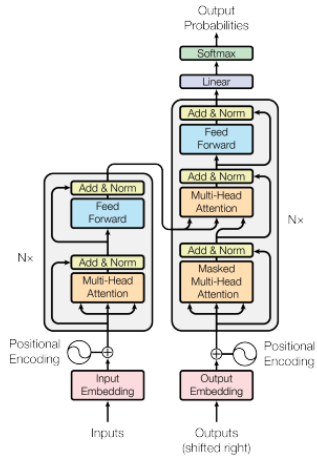


Figure 1: The Transformer - model architecture.

Ashish Vaswani et al (2017), *Attention is all you Need*

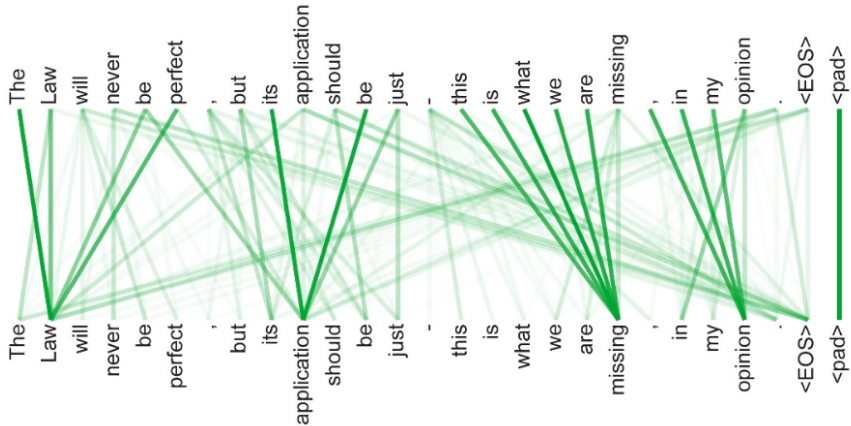
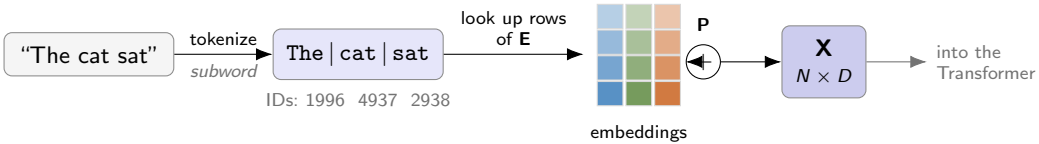


Figure 12.2 An example of learned attention weights. [From Vaswani *et al.* (2017) with permission.]

From Text to Vectors: Tokenization & Embeddings

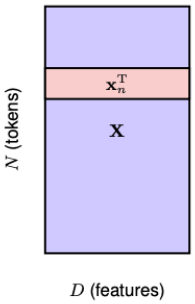


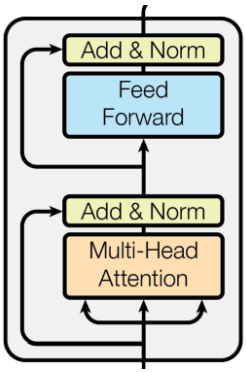
Tokens are subwords, not words (BPE / WordPiece / SentencePiece): a fixed vocabulary V that can spell *any* string, even unseen ones.

Output side (mirror): final vectors → Linear (weights often *tied* to $\mathbf{E}$) → softmax over V → next-token probabilities.

$\mathbf{E} \in \mathbb{R}^{V \times D}$ is **learned**; embeddings are usually scaled by $\sqrt{D}$ before adding $\mathbf{P}$.

Transformer Layer

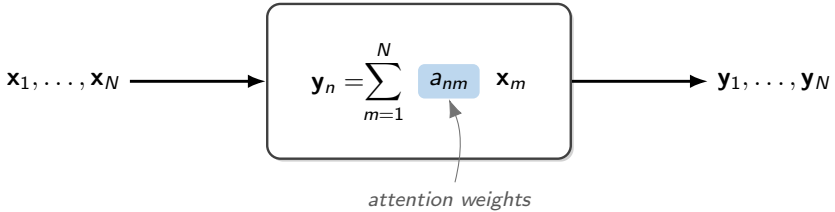


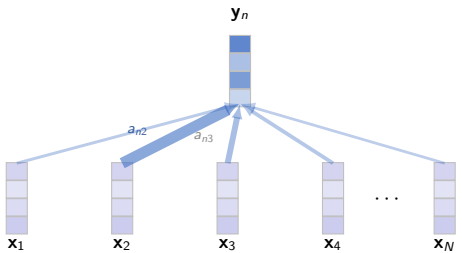


The structure of the data matrix $\mathbf{X}$, of dimension $N \times D$, in which row n represents the transposed data vector $\mathbf{x}_n^{\mathrm{T}}$.

$$\widetilde{\mathbf{X}} = \mathrm{TransformerLayer}\,[\mathbf{X}].$$

Attention





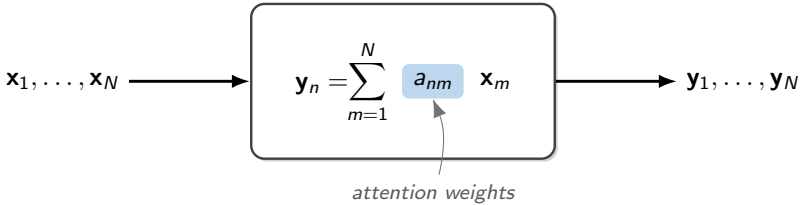
$$a_{nm} \geq 0$$

$$\sum_{m=1}^{N} a_{nm} = 1$$

a convex combination

Each output $\mathbf{y}_n$ is a weighted average of *all* inputs; the weights a_{nm} decide how much each $\mathbf{x}_m$ contributes.

Self-Attention



The weight is a *softmax over similarity scores* $\mathbf{x}_n^{\mathrm{T}}\mathbf{x}_m$ (a dot product):

$$a_{nm} = \mathrm{softmax}_m\left(\mathbf{x}_n^{\mathrm{T}}\mathbf{x}_m\right) = \frac{\exp\left(\mathbf{x}_n^{\mathrm{T}}\mathbf{x}_m\right)}{\sum_{m'=1}^{N}\exp\left(\mathbf{x}_n^{\mathrm{T}}\mathbf{x}_{m'}\right)}$$

$$\mathbf{Y} = \mathrm{Softmax}\left[\mathbf{X}\mathbf{X}^{\mathrm{T}}\right]\mathbf{X}$$

compact matrix form — all positions computed at once

Self-Attention as Information Retrieval

Role	Information retrieval (movie service)	Transformer self-attention	
Key k_n	each movie's attributes	input token x_n	$(k_n = x_n)$
Value v_n	the movie file (the payload)	the same token	$(v_n = x_n)$
Query q_m	the user's desired attributes	output-position token x_m	$(q_m = x_m)$
Score s_{mn}	match the query against every key	$s_{mn} = q_m^{\mathrm{T}} k_n$ (dot product)	
Weights a_{mn}	pick the single best match (*hard*)	$a_{mn} = \mathrm{softmax}_n(s_{mn})$ (*soft*)	
Output y_m	return the chosen movie	blend of values $y_m = \sum_n a_{mn} v_n$	

$$\mathbf{Y} = \mathrm{Softmax}\left[\mathbf{Q}\mathbf{K}^{\mathrm{T}}\right]\mathbf{V} \qquad \text{with} \qquad \mathbf{Q} = \mathbf{K} = \mathbf{V} = \mathbf{X}$$

Self-Attention as Information Retrieval: the flow



The query is compared with *every* key by a dot product $s_{mn} = q_m^{\mathrm{T}} k_n$; a **softmax** turns the scores into weights a_{mn}, and the output is the weighted blend of the values, $y_m = \sum_n a_{mn}\, v_n$.

Roadmap

From Recurrence to Attention

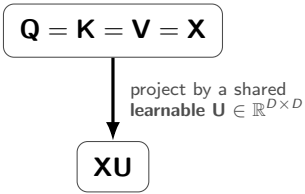
The Self-Attention Mechanism

Learning & Scaling Attention

The Transformer Model

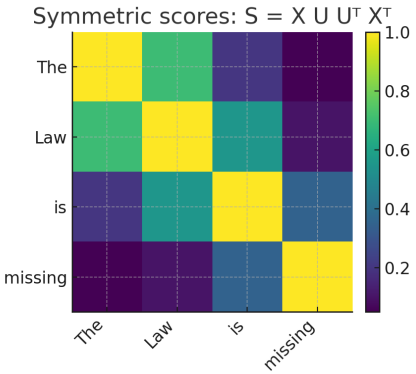
Training, Inference & LLMs

Learning the Attention (1)



$$\mathbf{Y} = \mathrm{Softmax}\left[\ \mathbf{XUU}^{\mathrm{T}}\mathbf{X}^{\mathrm{T}}\ \right]\mathbf{XU}$$

Problem. The score matrix $\mathbf{S} = \mathbf{XUU}^{\mathrm{T}}\mathbf{X}^{\mathrm{T}}$ is **symmetric** ($\mathbf{S} = \mathbf{S}^{\mathrm{T}}$): token *n* attends to *m* exactly as *m* attends to *n*. Real relations are *asymmetric* — we must break this.



The score matrix is mirror-symmetric across the diagonal.

Learning the Attention (2)

The fix: give each role its *own* learnable projection.

Query $\mathbf{Q} = \mathbf{X}\mathbf{W}^{(q)}$ $\quad \mathbf{W}^{(q)} : D \times D_k$

Key $\mathbf{K} = \mathbf{X}\mathbf{W}^{(k)}$ $\quad \mathbf{W}^{(k)} : D \times D_k$

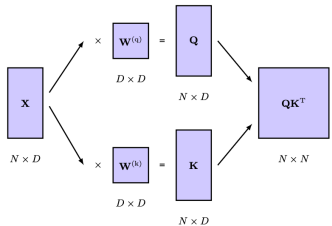
Value $\mathbf{V} = \mathbf{X}\mathbf{W}^{(v)}$ $\quad \mathbf{W}^{(v)} : D \times D_v$

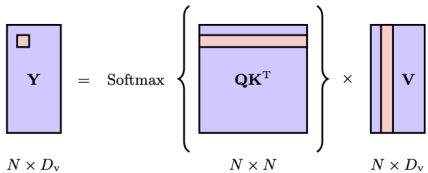
$$\mathbf{Y} = \mathrm{Softmax}\left[\mathbf{Q}\mathbf{K}^{\mathrm{T}}\right]\mathbf{V}$$

Fixed. $\mathbf{Q}$ and $\mathbf{K}$ use *different* matrices, so

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^{\mathrm{T}} = \mathbf{X}\mathbf{W}^{(q)}\left(\mathbf{W}^{(k)}\right)^{\mathrm{T}}\mathbf{X}^{\mathrm{T}}$$

is now **asymmetric**.





For convenience, we can set $D_k = D$ and $D_v = D$.

Scaled Self-attention

Problem. The logits are dot products $q_m^\top k_n$. If q, k have i.i.d. entries (mean 0, var 1),

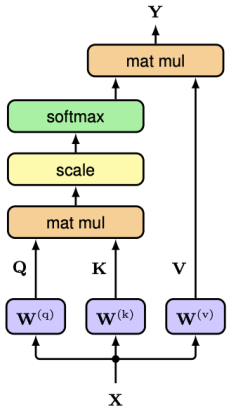
$$\mathbb{E}[q^\top k] = 0, \qquad \mathrm{Var}(q^\top k) = D_k$$

so their magnitude grows like $\sqrt{D_k}$. Large logits $\Rightarrow$ softmax saturates (near one-hot) $\Rightarrow$ **gradients vanish**.

Fix: divide the scores by $\sqrt{D_k}$ before the softmax.

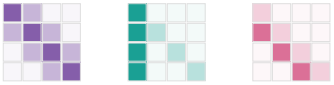
$$\mathrm{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathrm{Softmax}\Big(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{D_k}}\Big)\mathbf{V}$$

Effect. Keeps logits in a responsive range and stabilizes gradients — equivalently a softmax temperature $T = \sqrt{D_k}$. Scaling *preserves the ranking* of scores; it only tempers over-confidence.

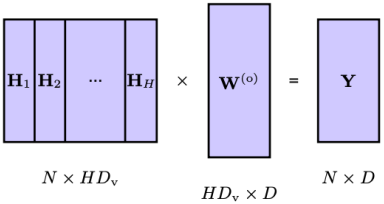


Multi-Head attention

One head must average competing patterns into one. Run H heads in parallel — each free to specialize:



Different heads → different attention patterns.



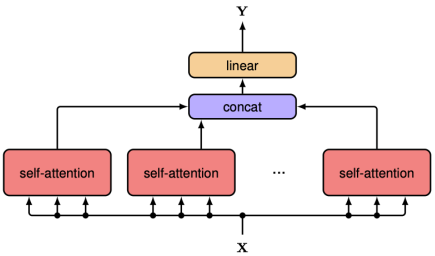
each head $h = 1, \ldots, H$:

$$\mathbf{Q}_h = \mathbf{X}\mathbf{W}_h^{(q)}, \quad \mathbf{K}_h = \mathbf{X}\mathbf{W}_h^{(k)}, \quad \mathbf{V}_h = \mathbf{X}\mathbf{W}_h^{(v)}$$

$$\mathbf{H}_h = \mathrm{Attention}(\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h) \quad (N \times D_v)$$

$$\mathbf{Y}(\mathbf{X}) = \underbrace{\mathrm{Concat}\left[\mathbf{H}_1, \mathbf{H}_2, \mathbf{H}_3, \ldots, \mathbf{H}_H\right]}_{N \times HD_v} \underbrace{\mathbf{W}^{(o)}}_{HD_v \times D}$$

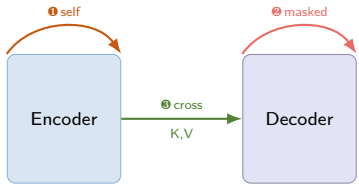
output $\mathbf{Y}$ has size $N \times D$.



Three Kinds of Attention

The computation is *always* $\mathrm{Softmax}\big(\mathbf{Q}\mathbf{K}^\top/\sqrt{D_k}\big)\mathbf{V}$ — only **where $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ come from** (and whether it is masked) changes.

Type	Q from	K,V from	Mask
encoder self-attention	enc	enc	–
decoder self-attention	dec	dec	**causal**
cross-attention (enc→dec)	dec	enc	–



❶ **Encoder self**: every token sees every token (bidirectional). ❷ **Decoder self**: causal — see only the past. ❸ **Cross**: the decoder *queries* the encoded source ($\mathbf{K}, \mathbf{V}$ from the encoder).

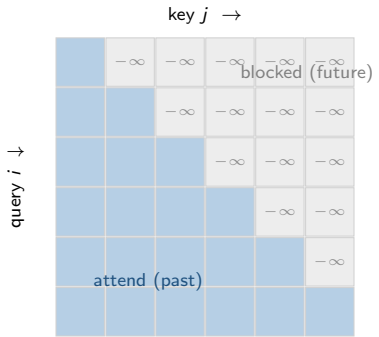
Causal (Masked) Self-Attention

Why? A decoder generates left-to-right. When predicting token $t{+}1$, position t may attend *only* to positions $\leq t$ — it must never peek at the future token it is trying to predict.

How? Add a mask to the scores *before* the softmax:

$$\text{Softmax}\Big(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{D_k}} + \mathbf{M}\Big)\mathbf{V}$$

$\mathbf{M}_{ij} = 0$ if $j \leq i$, else $-\infty$. The $-\infty$ entries become 0 after softmax — so future tokens get **zero** weight.



The attention matrix becomes **lower-triangular**.

Roadmap

From Recurrence to Attention

The Self-Attention Mechanism

Learning & Scaling Attention

The Transformer Model

Training, Inference & LLMs

The transformer Layers

Wrap *every* sub-layer the same way:

normalize each token across its features

$$\widetilde{\mathbf{x}} = \text{LayerNorm}\,\big(\,\mathbf{x} + \text{SubLayer}(\mathbf{x})\,\big)$$

residual (skip connection)

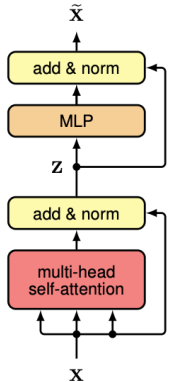
applied twice per layer:

$$\mathbf{Z} = \text{LayerNorm}\big[\,\mathbf{Y}(\mathbf{X}) + \mathbf{X}\,\big]$$

sub-layer 1: $\mathbf{Y}(\mathbf{X})$ = multi-head self-attention

$$\widetilde{\mathbf{X}} = \text{LayerNorm}\big[\,\text{MLP}[\mathbf{Z}] + \mathbf{Z}\,\big]$$

sub-layer 2: position-wise feed-forward (MLP)



Shape preserved ($N \times D \to N \times D$) — so layers stack freely.

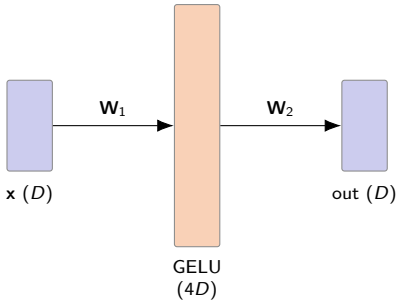
The Position-wise Feed-Forward Network

The second sub-layer: a small MLP applied to *each token vector* on its own.

$$\mathrm{FFN}(\mathbf{x}) = \mathbf{W}_2 \underbrace{\mathrm{GELU}(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)}_{\text{width } D_{\text{ff}}=4D} + \mathbf{b}_2$$

The complementary half. Attention mixes information *across tokens*; the FFN then processes *each token independently, across its features*. Same weights at every position.

Holds **$\sim$2/3 of a layer's parameters** ($\mathcal{O}(D \cdot D_{\text{ff}})$). Modern nets use GELU / SwiGLU activations.



expand $D \to 4D$, non-linearity, project $4D \to D$ — applied identically at all N positions.

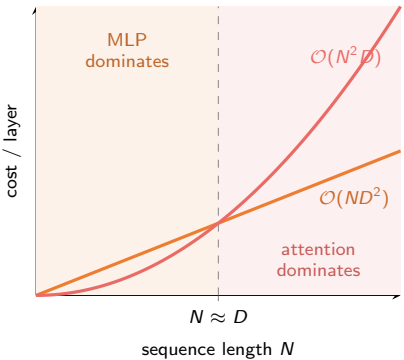
Computational Complexity

N tokens, width D ($D_k \simeq D_v \simeq D$); Q, K, V shared across tokens.

Component	Params	Compute
token-wise fully connected (baseline)	$\mathcal{O}(N^2D^2)$	$\mathcal{O}(N^2D^2)$
self-attention Q, K, V, W^O	$\mathcal{O}(D^2)$	$\mathcal{O}(N^2D)$
position-wise MLP	$\mathcal{O}(D^2)$	$\mathcal{O}(ND^2)$

total / layer: $\mathcal{O}(N^2D) + \mathcal{O}(ND^2)$

Vs. the fully-connected baseline $\mathcal{O}(N^2D^2)$, sharing Q, K, V across tokens makes the Transformer **far** cheaper — parameters drop to $\mathcal{O}(D^2)$.



Long sequences ($N \gg D$) → **attention** wins; wide models ($D \gg N$) → **MLP** wins.

Positional Encoding

Why? Self-attention is **permutation-invariant** — it sees a *set*, not a sequence, so *order is invisible*. We must inject position.

How? Add a fixed position pattern to every embedding:

$$\mathbf{X}_{\text{pos}} = \mathbf{X} + \mathbf{P}$$

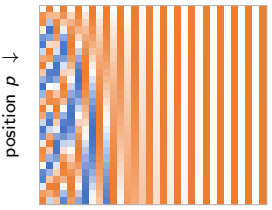
same $N \times D$ shape

Sinusoidal encoding (parameter-free), for feature pair $2i, 2i{+}1$:

$$\mathrm{PE}(p, 2i) = \sin\left(p/10000^{2i/D}\right)$$
$$\mathrm{PE}(p, 2i{+}1) = \cos\left(p/10000^{2i/D}\right)$$

✓ absolute position ✓ relative distance

✓ no parameters ✓ extrapolates to longer N



Each **row** is one position's fingerprint. Low dims oscillate fast, high dims slowly ⇒ relative offsets become smooth phase shifts.

Modern variants: *learned* & *relative* encodings; **RoPE** (rotary — today's LLMs) and **ALiBi** — better length extrapolation.

The transformer architecture

- Embedding to encode text
- Positional encoding to preserve the sequence order
- Multi-head self-attention for
  - Parallel computation
  - Multi-view learning
- Add & Normalize
  - Residual connection followed by layer normalization (not batchnorm)
- Position-wise feed-forward network
  - Pool/aggregate the "learned" information for each position
- Masking: each position in the decoder is allowed to only attend to all positions in the decoder up to (and including) that position

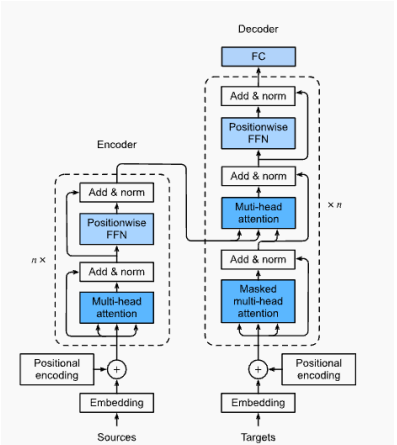


Fig. 11.7.1 The Transformer architecture.

Aston Zhang, Zachary Lipton, Mu Li, and Alex Smola (2023) *Dive into Deep Learning*

Roadmap

From Recurrence to Attention

The Self-Attention Mechanism

Learning & Scaling Attention

The Transformer Model

Training, Inference & LLMs

Three Transformer Families

Same block, three ways to wire attention — each with its own pretraining objective.

Encoder-only (BERT)
- attention: **bidirectional**
- pretrain: **masked LM** (fill in blanked tokens)
- use: understanding, embeddings, classification

Decoder-only (GPT, LLaMA)
- attention: **causal**
- pretrain: **next-token** (predict the future)
- use: generation, chat, in-context learning

Encoder–decoder (T5)
- attention: bi + causal + **cross**
- pretrain: span corruption / seq2seq
- use: translation, summarization

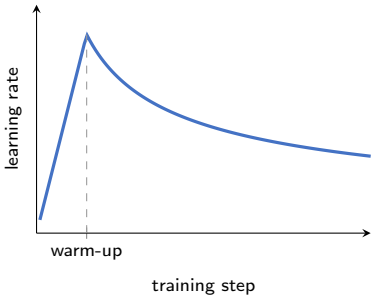
Pretrain then adapt: self-supervised pretraining on huge unlabeled corpora, then fine-tuning or *in-context learning* (prompting) for downstream tasks.

Training a Transformer

Objective. Cross-entropy on the next token; *teacher forcing* feeds the ground-truth prefix so all positions train in parallel.

Optimizer. Adam (β_1=0.9, β_2=0.98) with the signature **warm-up then inverse-$\sqrt{\ }$ decay** learning-rate schedule.

Regularization. Dropout ($\sim$0.1) on attention, residuals & embeddings; **label smoothing** ($\sim$0.1); weight decay.

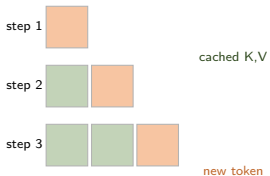


Linear warm-up, then decay $\propto 1/\sqrt{\text{step}}$.

Decoding & the KV Cache

Autoregressive generation: predict a token, append it, repeat.

Strategy	Picks the next token by...
greedy	the single most likely token
beam search	keeping the best few running sequences
sampling	drawing from the distribution (temperature, top-k, top-p/nucleus)



KV cache: store past keys & values, so each new token costs $\mathcal{O}(N)$, not $\mathcal{O}(N^2)$. This is what makes generation fast.

Scaling & Efficiency (Beyond the Basics)

The bottleneck. Self-attention is $\mathcal{O}(N^2)$ in time *and* memory — the wall that limits context length.

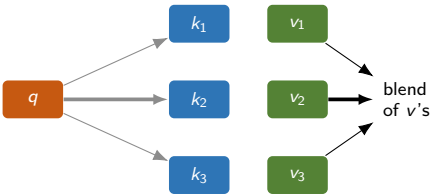
How modern systems push past it:

- **FlashAttention** — *exact*, IO-aware; same result, far less memory traffic.
- **Sparse / linear attention** (Longformer, Performer) — *approximate*, sub-quadratic.
- **MQA / GQA** — share $\mathbf{K}, \mathbf{V}$ across heads $\Rightarrow$ smaller KV cache.
- **Pre-LN & RMSNorm** — stable training at great depth.

Scaling laws. Loss falls as a smooth power law in parameters, data, and compute — the empirical basis for building ever-larger models.

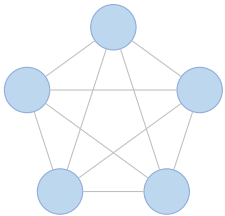
Two Mental Models

Attention = soft dictionary lookup



A query retrieves a *weighted blend* of values — a differentiable, fuzzy hash table.

Transformer = message passing on a graph



Tokens are **nodes**, attention is **learned edges**, the FFN is the **node update**.

Change *what the graph connects* and the same machinery applies — words, **time steps (Part 2)**, or **table columns (Part 3)**.

Genealogy of Transformer Based LLMs

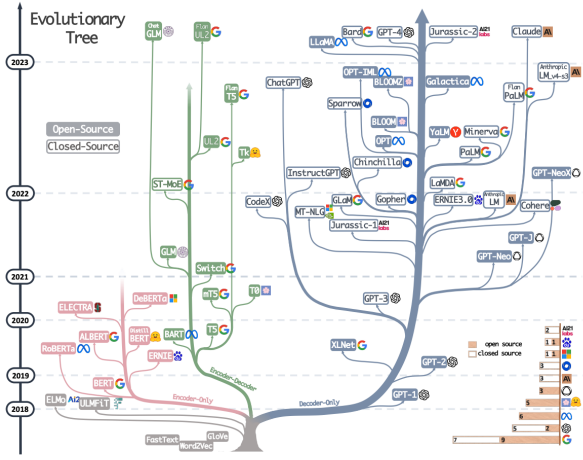


Fig. 1. The evolutionary tree of modern LLMs traces the development of language models in recent years and highlights some of the most well-known models. Models on the same branch have closer relationships. Transformer-based models are shown in non-grey colors: decoder-only models in the blue branch, encoder-only models in the pink branch, and encoder-decoder models in the green branch. The vertical position of the models on the timeline represents their release dates. Open-source models are represented by solid squares, while closed-source models are represented by hollow ones. The stacked bar plot in the bottom right corner shows the number of models from various companies and institutions.

References

Foundational papers

- Cho et al. (2014), Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation.
- Sutskever et al. (2014), Sequence to Sequence Learning with Neural Networks.
- Bahdanau et al. (2014), Neural Machine Translation by Jointly Learning to Align and Translate.
- Vaswani et al. (2017), Attention Is All You Need.

Textbooks

- Uday Kamath et al., Transformers for Machine Learning: A Deep Dive.
- Charu C. Aggarwal, Neural Networks and Deep Learning: A Textbook.
- Zhang, Lipton, Li, Smola (2023), Dive into Deep Learning.